

Skip List (Advanced Data structure)

As we know that we have basic data structure and the problems in the world are what inspires us to combine the concepts of data structures to find solution for problems we face. So if we want to find a **Simple Dynamic Research Data Structure** . What does this even mean?

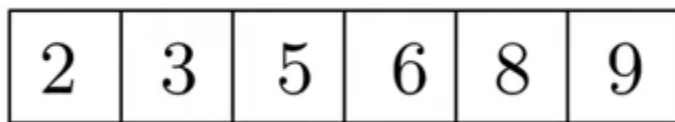
Dynamic⇒ Efficient update such as insert and delete.

Search⇒ find element , predecessor and successor.

So the first thing that comes in mind is Array ⇒

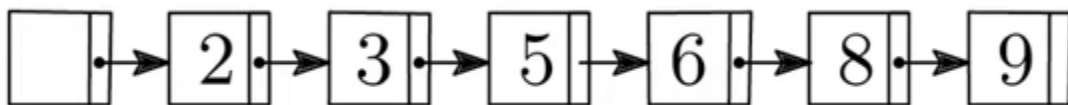
The sorted array search is $O(\log(n))$ and update is $O(n)$

So Searching is fine , but updating takes a linear time which is not quite good for the best case.



So what about linked list we can use it ⇒

The sorted linked list search is $O(n)$ and update is $O(1)$ as we search first and then update so it's constant time. However the search is a linear time also .

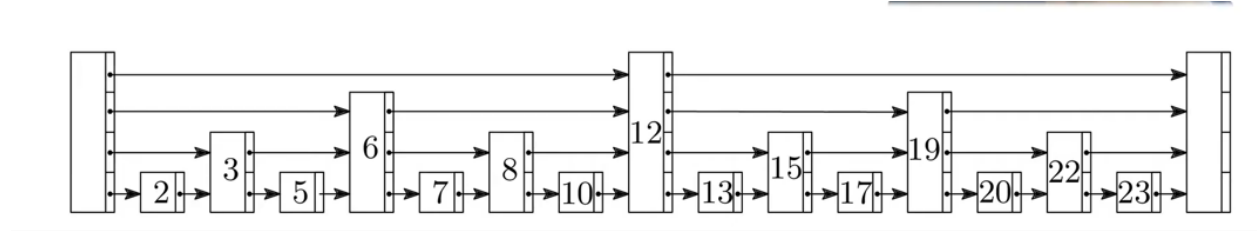


So what if we combined both data structures to have a more efficient data structure with simple dynamic search ? That combination results into a Skip List .

so what is the idea of a Skip List? it's basically a combination of array and linked list which has linked levels. But what does linked levels mean? It's main idea is :

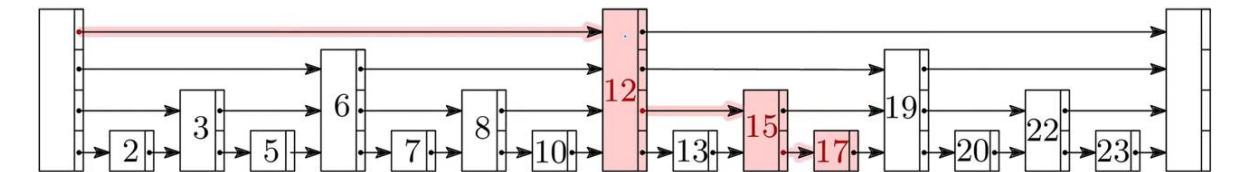
- To keep a sorted linked list to have an update time $O(1)$.
- To add shortcuts between elements which means to skip element .
- Add shortcuts between shortcuts.

this is an illustration of it :



How does it work?

for example we want to search for $x=17$ in this skip list :



First we search in the top shortcut is 17 greater than or less than 12 ? The answer is greater than so we search at the part greater than 12 .

Secondly we compare in the second shortcut which is 19 is 17 greater than or less than 19? the answer is less than so we search at the less part.

Now we go to the first level of shortcuts if 17 greater than or less than 15 it's greater than and Boom we found it !!!

The number of levels is $\log(n)$ so the search time is $\log(n)$ which is efficient.

This is how search is done , but how can we update let's take a look about how we insert.

- We first have to search for predecessor in all levels (because we keep a sorted linked list)
- Randomize to choose elements for the levels .

What does randomize means $\Rightarrow$ it means we don't have a pattern in choosing what element will be the shortcut we make it random for efficient and not fixed structure .

The same goes with deleting we need to randomize first. This is a simple illustration of how to randomize:

- every element in the bottom level
-  $p=1/2$: element also in the next level
-  $p=1/2$: element in no higher level

so head means higher and tails mean stop going higher for this element so probability is 1/2 for each one.

What is the expected size of a skip list ?

- all: $n \cdot \sum_{i=0}^{\infty} 1/2^i = 2n$

$\Rightarrow$ expected size of skip list = $O(n)$

But the update time is the same as the search time $\log(n)$.

We can alternatively use the binary search tree ,but it's more complicated . So this means it's used for scenarios that need fast , search , insertion and deletion example:

Priority Queues

- Skip lists can implement **priority queues** where the highest-priority element is always at the front.
- Faster than binary heaps for some operations (e.g., deletions in the middle).

Example:

- Used in some task schedulers where dynamic priority updates are needed.

This is priority queue:

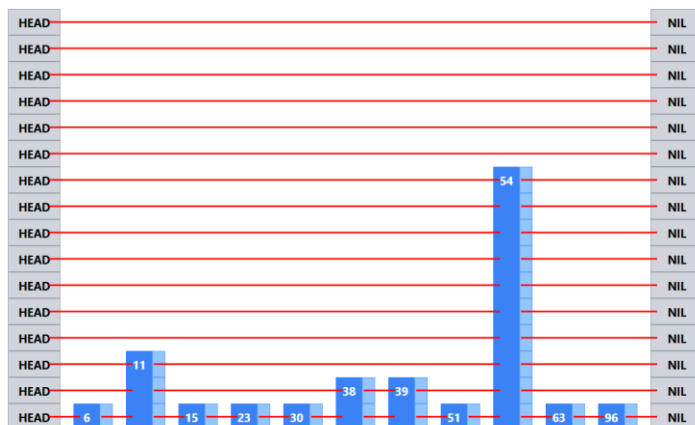
Queue

2	10	5	20	25	30	15
10	A	D	B	E	C	F

Visualization of Skip List :

Skip List `</>` [←](#)

Operation: Insert 15, Result: Insertion successful



Settings

Max Level: 16

Probability: 1/2

Random Init

15

Insert Search Delete

Visualization reference: <https://gallery.selfboot.cn/en/algorithms/skiplist>